

zapperV9-03

Zv09.03

Generated by Doxygen 1.8.16

Chapter 1

Adafruit INA219 current/power monitor IC

1.1 Introduction

Driver for the INA219 current sensor

This is a library for the Adafruit INA219 breakout —> <https://www.adafruit.com/products/904>

Adafruit invests time and resources providing this open source code, please support Adafruit and open-source hardware by purchasing products from Adafruit!

1.2 Author

Written by Kevin "KTOWN" Townsend for Adafruit Industries.

1.3 License

BSD license, all text here must be included in any redistribution.

Chapter 2

Adafruit INA219 Library `<a href="https://travis-ci.org/adafruit/Adafruit_INA219"></a>`

This is a library for the Adafruit INA219 high side DC current sensor boards:

- <https://www.adafruit.com/products/904>
- <https://www.adafruit.com/product/3650>

Check out the links above for our tutorials and wiring diagrams. This chip uses I2C to communicate

2.1 Compatibility

MCU	Tested	Works	Doesn't Work	Not Tested	Notes
Atmega328 @ 16MHz	X				
Atmega328 @ 12MHz	X				
Atmega32u4 @ 16MHz	X				
Atmega32u4 @ 8MHz	X				
ESP8266	X				
Atmega2560 @ 16MHz	X				
ATSAM3X8E	X				
Use D20/D21. ATSAM21D	X				
ATtiny85 @ 16MHz	X				
Use SDA/SCL D0/D2 ATtiny85 @ 8MHz	X				
Use SDA/SCL D0/D2 Intel Curie @ 32MHz	X				
STM32F2	X				

- ATmega328 @ 16MHz : Arduino UNO, Adafruit Pro Trinket 5V, Adafruit Metro 328, Adafruit Metro Mini
- ATmega328 @ 12MHz : Adafruit Pro Trinket 3V
- ATmega32u4 @ 16MHz : Arduino Leonardo, Arduino Micro, Arduino Yun, Teensy 2.0
- ATmega32u4 @ 8MHz : Adafruit Flora, Bluefruit Micro
- ESP8266 : Adafruit Huzzah
- ATmega2560 @ 16MHz : Arduino Mega
- ATSAM3X8E : Arduino Due

- ATSAM21D : Arduino Zero, M0 Pro
- ATtiny85 @ 16MHz : Adafruit Trinket 5V
- ATtiny85 @ 8MHz : Adafruit Gemma, Arduino Gemma, Adafruit Trinket 3V

Adafruit invests time and resources providing this open source code, please support Adafruit and open-source hardware by purchasing products from Adafruit!

Written by Ktown for Adafruit Industries. MIT license, all text above must be included in any redistribution

Chapter 3

Module Index

3.1 Modules

Here is a list of all modules:

Sistema-hardware	11
Rife-Clark-Zappicator	18
Interface-com-Ina219	28
Nextion-HMI-Display	31

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

[Adafruit_INA219](#)

Class that stores state and functions for interacting with INA219 current/power monitor IC . . . [55](#)

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

C:/Users/mario/Documents/Arduino/ESP32_zapperV9.0/ ESP32_zapperV9.0.ino	
Modulo Esp32-zapper v 9.0 funcoes de tratamento do PWM para zapper, clark, rife, zappicator. funcoes da tela HMI - Nextion. funcoes de leitura do INA219 - sensor tensao e corrente	??
C:/Users/mario/Documents/Arduino/ESP32_zapperV9.0/___vm/. ESP32_zapperV9.0.vsarduino.h	??
C:/Users/mario/Documents/Arduino/ESP32_zapperV9.0/arquivos-dox/ Arduino.h	61
C:/Users/mario/Documents/Arduino/ESP32_zapperV9.0/arquivos-dox/ mario_nextion.h	??
C:/Users/mario/Documents/Arduino/ESP32_zapperV9.0/arquivos-dox/ Nextion.h	??
C:/Users/mario/Documents/Arduino/ESP32_zapperV9.0/arquivos-dox/ tabelas_rife-esp32.h	??
C:/Users/mario/Documents/Arduino/ESP32_zapperV9.0/arquivos-dox/Adafruit_INA219/ Adafruit_INA219.h	

Chapter 6

Module Documentation

6.1 Sistema-hardware

rotinas-funcoes relacionadas ao hardware ESP32 WROOM-32

Functions

- void `setup` ()
inicializacao do zapper V9.0
- void `loop` ()
Loops this object.
- void `daBeep` (int tempoBip)

*****DEFINICOES/PROTOTOPOS*****
- void `avisaXseg` (int tempoBip, int numeroBips)

***** AVISO DE N BEEPS POR X SEG ***** 1 segundo entre bips (1000 ms) ***cuidado com rotina para nao travar processamento
- void `tickRelogio` ()
relogio feito por software no loop - nao vai usar
- void `iniciaDebug` ()
envia msg ao monitor inicio do debug
- void `dbgMostraDados` (String mensagem, int valor)
mostra msg e dados - monitor
- void `iniciaPWMs` ()
inicia ondas quadradas dos protocolos-tratamentos
- void `terminaPrograma` (int protocolo)
termina protocolo-tratamento
- void `ligaPWM` (float pwmFreq)
liga pwm
- void `desligaPWM` ()
desliga pwm
- void `terminaProgramaZ` (int protocolo)
termina protocolo tipo zappicator
- void `ligaPWMZ` (float entraFreq)
liga pwm do tipo zappicator
- void `desligaPWMZ` ()
termina pwm do tipo zappicator

6.1.1 Detailed Description

rotinas-funcoes relacionadas ao hardware ESP32 WROOM-32

6.1.2 Function Documentation

6.1.2.1 `avisaXseg()`

```
void avisaXseg (
    int tempoBip,
    int numeroBips )
```

***** AVISO DE N BEEPS POR X SEG ***** 1 segundo entre bips (1000 ms) ***cuidado com rotina para nao travar processamento

Date

10/12/2019

Parameters

<i>tempoBip</i>	int; The tempo bip (ms)
<i>numeroBips</i>	int; The numero bips.

< numeroBips - quantos bips

<um bip - tempoBip de um bip

< 0,5 segundo entre bips

6.1.2.2 `daBeep()`

```
void daBeep (
    int tempoBip )
```

*****DEFINICOES/PROTOTIPOS *****

da um BEEP (tempo em milisegundos)

bipa por(miliseg) definido em "int"

Date

10/12/2019

Parameters

<i>tempoBip</i>	int; The tempo bip em milisegundos.
-----------------	-------------------------------------

< bit, frequencia, tempo ms

6.1.2.3 dbgMostraDados()

```
void dbgMostraDados (
    String mensagem,
    int valor )
```

mostra msg e dados - monitor

Debug mostra dados.

Date

10/12/2019

Parameters

<i>mensagem</i>	The mensagem (String).
<i>valor</i>	The valor (int).

< mensagem para monitor

< valor (int) para monitor

6.1.2.4 desligaPWM()

```
void desligaPWM ( )
```

desliga pwm

Desliga pwm.

Date

10/12/2019

6.1.2.5 desligaPWMZ()

```
void desligaPWMZ ( )
```

termina pwm do tipo zappicator

Desliga pwm do programa zappicator.

Date

10/12/2019

6.1.2.6 iniciaDebug()

```
void iniciaDebug ( )
```

envia msg ao monitor inicio do debug

Inicia debug, envia msg para monitor (pela USB)

Date

10/12/2019

6.1.2.7 iniciaPWMs()

```
void iniciaPWMs ( )
```

inicia ondas quadradas dos protocolos-tratamentos

Inicia PWMs, clark-rife e zappicator sao dois canais (bits/pinos)

Date

10/12/2019

6.1.2.8 ligaPWM()

```
void ligaPWM (
    float pwmFreq )
```

liga pwm

Liga pwm.

Date

10/12/2019

Parameters

<i>pwmFreq</i>	float; (frequencia de saida).
----------------	-------------------------------

< 1 bit duty

< 0 ou 1 = 50% duty

< 8 bits duty

< 127 = 50% duty

6.1.2.9 ligaPWMZ()

```
void ligaPWMZ (
    float entraFreq )
```

liga pwm do tipo zappicator

Liga pwm do programa zappicator.

Date

10/12/2019

Parameters

<i>entraFreq</i>	float; frequencia de trabalho.
------------------	--------------------------------

6.1.2.10 loop()

```
void loop (
    void )
```

Loops this object.

Date

10/12/2019

< controle do protocolo Clark

< — controle do Clark - 7 min

< — controle do Clark 20min

< — termina Clark se fim

< controle do protocolo zappicator

< controle do protocolo Rife pre-definido

< controle do protocolo Rife com N. do protocolo

< controle da entrada manual de frequencias Rife (6 freq+duracao)

< le sensor Ina219 - corrente e tensao

6.1.2.11 setup()

```
void setup (  
    void )
```

inicializacao do zapper V9.0

Date

10/12/2019

6.1.2.12 terminaPrograma()

```
void terminaPrograma (  
    int protocolo )
```

termina protocolo-tratamento

Termina programa-protocolo, limpa status e da uns(5) beeps.

Date

10/12/2019

Parameters

<i>protocolo</i>	int; protocolo.
------------------	-----------------

< deve faltar "limpar" mais flags e Ns

6.1.2.13 terminaProgramaZ()

```
void terminaProgramaZ (  
    int protocolo )
```

termina protocolo tipo zappicator

Termina programa zappicator (rotinas timer 1).

Date

10/12/2019

Parameters

<i>protocolo</i>	int; qual protocolo.
------------------	----------------------

< deve faltar "limpar" mais flags e Ns

6.1.2.14 tickRelogio()

```
void tickRelogio ( )
```

relogio feito por software no loop - nao vai usar

Tick relógio feito por software usando o loop (pessima ideia) mudado para RTC-DS1307+relogio interno do sistema, isto, na versao/modulo AsensoresRTC-BDV7.ino.

Date

10/12/2019

6.2 Rife-Clark-Zappicator

rotinas-funcoes relacionadas aos protocolos-tratamentos Rife, Clark, Zappicator a)descricao linha da zappicator {n, x,y,x,y,x,y,...} n - quantidade de grupos frequencia+tempo ligada, x-frequencia, (em Hz) y-tempo ligada a freq. (em segundos) -> 15min = 900seg o original e uma frequencia 1KHz e uns +/- 15min (tembem li 30m) aqui pode colocar mais freq igual a clark "pares"=freq+tempo

Functions

- void `limpaStatus` (int protocolo)
limpa todos flags do protocolo em execucao
- void `tempoMarcaInicio` ()
guarda hora que comecou protocolo
- unsigned long `tempoLeAgora` ()
retorna false se estourou tempo
- void `comecaClark` ()
Inicia protocolo CLARK.
- void `proximoParClark` ()
Proximo par clark.
- void `clark_7min` ()
Controle de tempo (7 min) - CLARK.
- void `clark_20min` ()
Controle de tempo (20 min) - Clark.
- void `avancaBarraClark` (int incremento)
Avanca barra clark.
- void `comecaZappic` ()
comeca ZAPPICATOR
- void `controleZappic` ()
Controle do zappicator.
- void `avancaBarraZapp` (float tempoDivisor)
Avanca barra zapp.
- void `comecaRifePre` (int `qualRife`)
Comeca RIFE PRE.
- void `proximoRifePre` ()
Proximo rife pre.
- void `avancaBarraRifePre` (float tempoDivisor)
ALGO ESTRANHO COM ESTA AVANCA BARRAS DO RIFE.
- void `controleRifePre` ()
Controle rife pre.
- void `comecaRifeETDFL` (int `qualRife`)
RIFE ETDFL.
- void `proximoRifeETDFL` ()
Proximo rife etdfl.
- void `avancaBarraRifeETDFL` (float tempoDivisor)
Avanca barra rife etdfl.
- void `controleRifeETDFL` ()
Controle rife etdfl.
- void `comecaRifeManual` ()
*Comeca RIFE MANUAL vai ler frequencias e tempos; numero lido - getvalue - usa little endian order 0x01, 0x02, 0x03,0x04 (4 bytes na msg de retorno) 0x01+(0x02*256)+(0x03*65536)+(0x04*16777216)*

- void `proximoRifeManual` ()
Proximo rife manual.
- void `avancaBarraRifeManual` (float tempoDivisor)
Avanca barra rife manual.
- void `controleRifeManual` ()
Controle rife manual.

Variables

- bool `emPrograma` = false
****** flag de em programa de Rife, Clark, Zappicator*
- int `qualPrograma` = 0
identifica programa, 1-clark, 2-zappic, 3-rife, etc)
- int `estagio` = 0
indica em que estagio esta - qual par de frequencias
- int `quantosEstagios` = 0
indica quantos estagios-fases tem o protocolo
- float `pwmFrequencia` = 0
indica a (pwm) frequencia
- float `pwmDuracao` = 0
indica (pwm) a duracao do protocolo-fase-estagio
- bool `primeiraVez` = true
indica se comecou(1 vez) CLARK
- unsigned long `protocoloAntesRasc`
do protocolo CLARK antes rascunho
- unsigned long `protocoloSegundosTotal` = 0
do protocolo CLARK contador segundos total
- unsigned long `protocoloSegundos` = 0
do protocolo CLARK contador segundos
- int `qualRife` = 1
indica qual protocolo Rife
- uint32_t `frequenciaRasc` [7]
Operacao Manual (Rife) tabela de rascunho das frequencias.
- uint32_t `duracaoRasc` [7]
Operacao Manual (Rife) tabela de duracao das frequencias.

6.2.1 Detailed Description

rotinas-funcoes relacionadas aos protocolos-tratamentos Rife, Clark, Zappicator a)descricao linha da zappicator {n, x,y,x,y,x,y,....} n - quantidade de grupos frequencia+tempo ligada, x-frequencia, (em Hz) y-tempo ligada a freq. (em segundos) -> 15min = 900seg o original e uma frequencia 1KHz e uns +/- 15min (tambem li 30m) aqui pode colocar mais freq igual a clark "pares"=freq+tempo

b)descricao linha da CLARK {n,x,y,x,y,x,y,....} n - quantidade de grupos frequencia+tempo ligada, x-frequencia, (em Hz) y-tempo ligada a freq. (em segundos) Clark "oficial" (5 fases/etapas): 30KHz->7min;descanco->20min;30KHz->7min;descanco->20min;30KHz->7min;FIM

c)CAFL-ETDFL-2018 RIFE: {x,y,t, zzz,zzz,zzz,.....} x - numero do programa, y - quantidade de frequencias, t - tempo para cada frequencia em segundos-por enquanto,(minx60) zzz- as frequencias em *** KHz *** TRA↔NSFERI TABELAS RIFE PARA => [tabelas_rife-ESP32.h](#) (esta nos "include") olhar comentarios la e tambem "a fazer".

6.2.2 Function Documentation

6.2.2.1 `avancaBarraClark()`

```
void avancaBarraClark (  
    int incremento )
```

Avanca barra clark.

Date

10/12/2019

Parameters

<i>incremento</i>	int; incremento.
-------------------	------------------

6.2.2.2 `avancaBarraRifeETDFL()`

```
void avancaBarraRifeETDFL (  
    float tempoDivisor )
```

Avanca barra rife etdfl.

Date

10/12/2019

Parameters

<i>tempoDivisor</i>	float; tempo divisor.
---------------------	-----------------------

6.2.2.3 `avancaBarraRifeManual()`

```
void avancaBarraRifeManual (  
    float tempoDivisor )
```

Avanca barra rife manual.

Date

10/12/2019

Parameters

<i>tempoDivisor</i>	float; The tempo divisor.
---------------------	---------------------------

6.2.2.4 avancaBarraRifePre()

```
void avancaBarraRifePre (  
    float tempoDivisor )
```

ALGO ESTRANHO COM ESTA AVANCA BARRAS DO RIFE.

Date

10/12/2019

Parameters

<i>tempoDivisor</i>	float ;divisor de tempo para inc da barra.
---------------------	--

6.2.2.5 avancaBarraZapp()

```
void avancaBarraZapp (  
    float tempoDivisor )
```

Avanca barra zapp.

Date

10/12/2019

Parameters

<i>tempoDivisor</i>	float; The tempo divisor.
---------------------	---------------------------

6.2.2.6 clark_20min()

```
void clark_20min ( )
```

Controle de tempo (20 min) - Clark.

Date

10/12/2019

< se terminou uma etapa avanca barra no display

6.2.2.7 clark_7min()

```
void clark_7min ( )
```

Controle de tempo (7 min) - CLARK.

Date

10/12/2019

< + 2

< (6 x 3) para resultar "20"

6.2.2.8 comecaClark()

```
void comecaClark ( )
```

Inicia protocolo CLARK.

Date

10/12/2019

< indica que programa Clark começa

< indica que 1 estagio começa

< tempo em segundos

6.2.2.9 comecaRifeETDFL()

```
void comecaRifeETDFL (
    int qualRife )
```

RIFE ETDFL.

Date

10/12/2019

Parameters

<i>qualRife</i>	int ;qual o programa Rife.
-----------------	----------------------------

< indica que programa RIFE comeca

< indica que 1 estagio comeca

<ERRO

<inicia display do nextion

< INCONSISTENTE LOGICA.

6.2.2.10 comecaRifeManual()

```
void comecaRifeManual ( )
```

Comeca RIFE MANUAL vai ler frequencias e tempos; numero lido - getvalue - usa little endian order 0x01, 0x02, 0x03,0x04 (4 bytes na msg de retorno) 0x01+(0x02*256)+(0x03*65536)+(0x04*16777216)

Date

10/12/2019

< indica que o programa RIFE comeca

< indica que 1 estagio comeca

vai ler frequencias e tempos; numero lido - getvalue - usa little endian order 0x01,0x02,0x03,0x04 (4 bytes na msg de retorno) 0x01+(0x02*256)+(0x03*65536)+(0x04*16777216)

< se nao houver freq1 ou dura1 cai fora

< inicia display do nextion

6.2.2.11 comecaRifePre()

```
void comecaRifePre (
    int qualRife )
```

Comeca RIFE PRE.

Date

10/12/2019

Parameters

<i>qualRife</i>	int; qual programa rife.
-----------------	--------------------------

< indica que programa RIFE comeca

< indica que 1 estagio comeca

< ERRO

< inicia display do nextion

6.2.2.12 **comecaZappic()**

```
void comecaZappic ( )
```

comeca ZAPPICATOR

Date

10/12/2019

< indica que programa Zapiccatior começa

< indica que 1 estágio começa

< CUIDADO-PROVISORIO-TT

< ve se liga 1 KHz

< ve se liga 2.5 KHz

< ve se liga 428 Hz

< ve se liga 529 Hz

< ve qual tempo de duração

< converte min para seg

< zera barra no display

< liga pwm

< desliga pwm

6.2.2.13 **controleRifeETDFL()**

```
void controleRifeETDFL ( )
```

Controle rife etdfl.

Date

10/12/2019

<controla (escala) avanço de barras

6.2.2.14 controleRifeManual()

```
void controleRifeManual ( )
```

Controle rife manual.

Date

10/12/2019

< controla (escala) avanço de barras

6.2.2.15 controleRifePre()

```
void controleRifePre ( )
```

Controle rife pre.

Date

10/12/2019

6.2.2.16 controleZappic()

```
void controleZappic ( )
```

Controle do zappicator.

Date

10/12/2019

< controla (escala) avanço de barras

6.2.2.17 limpaStatus()

```
void limpaStatus (
    int protocolo )
```

limpa todos flags do protocolo em execucao

Limpa status: zera variaveis dos programas/protocoloes e frequencias. Identifica o protocolo por causa do rife manual para limpar tabela rascunho.

Date

10/12/2019

Parameters

<i>protocolo</i>	int; protocolo (identifica qual protocolo que tem que parar).
------------------	---

6.2.2.18 proximoParClark()

```
void proximoParClark ( )
```

Proximo par clark.

Date

10/12/2019

< para controlar rotinas de clark7min e clark20m

6.2.2.19 proximoRifeETDFL()

```
void proximoRifeETDFL ( )
```

Proximo rife etdfl.

Date

10/12/2019

<inicia display do nextion

6.2.2.20 proximoRifeManual()

```
void proximoRifeManual ( )
```

Proximo rife manual.

Date

10/12/2019

6.2.2.21 proximoRifePre()

```
void proximoRifePre ( )
```

Proximo rife pre.

Date

10/12/2019

< inicia display do nextion

6.2.2.22 tempoLeAgora()

```
unsigned long tempoLeAgora ( )
```

retorna false se estourou tempo

Le tempo que passou desde inicio do programa retorna false se estourou tempo. Verifica o tempo que timer-pwm esta ligado/desligado, em funcao do programado em "tempoReferencia" - em segundos. O tempoInicial foi quando comecou (na rotina tempoMarcaInicio). O rasc o tempo(relogio) e agora. O tempoReferencia e quanto tem que demorar.

Date

10/12/2019

Returns

unsigned long diferenca (retorna quanto tempo passou, em segundos)

< para ter em segundos divide por 1000

< volta tempo decorrido em segundos

6.2.2.23 tempoMarcaInicio()

```
void tempoMarcaInicio ( )
```

guarda hora que comecou protocolo

marca tempo inicial do comeco do protocolo e flags de primeira vez, antes, para leitura do ina219, etc.

Date

10/12/2019

< marca qdo comecou em milissegundos

6.3 Interface-com-Ina219

rotinas-funcoes de tratamento da leitura de tensao e corrente do Ina219

Functions

- void [inicialINA219](#) ()
inicia chip sensores de tensao e corrente ina219
- void [leINA219](#) ()
le sensores-entradas analogicas de tensao e corrente do ina219
- void [dbgMostraIna219](#) (float [voltageCircuito](#), float [voltageShunt](#), float [voltageEntrada](#), float [corrente_mA](#), float [potencia_mW](#), float [mediaVolts](#), float [mediaAmperes](#))
Debug mostra valores ina 219 (mA, mV, mW, media Volt, media A)

Variables

- [Adafruit_INA219](#) [ina219](#)
cria instancia do INA219 - [Adafruit_INA219](#) ina219
- unsigned long [tempoIna219Rasc](#) = 0
unsigned long, rascunho de tempo-timer do Ina219
- int16_t [n_ina](#) = 0
rascunho de valor do ina219
- float [mediaVolts](#) = 0
media de volts medidos/lidos
- float [mediaAmperes](#) = 0
media de amperes medidos/lidos
- float [accVolts](#) = 0
acumulado de volts
- float [accAmperes](#) = 0
acumulado de amperes
- bool [paraLeitura](#) = false
para temporariamente leitura

6.3.1 Detailed Description

rotinas-funcoes de tratamento da leitura de tensao e corrente do Ina219

6.3.2 Function Documentation

6.3.2.1 dbgMostraIna219()

```
void dbgMostraIna219 (
    float voltagemCircuito,
    float voltagemShunt,
    float voltagemEntrada,
    float corrente_mA,
    float potencia_mW,
    float mediaVolts,
    float mediaAmperes )
```

Debug mostra valores ina 219 (mA, mV, mW, media Volt, media A)

Date

10/12/2019

Parameters

<i>voltagemCircuito</i>	float;voltagem circuito.
<i>voltagemShunt</i>	float;voltagem shunt.
<i>voltagemEntrada</i>	float;voltagem entrada.
<i>corrente_mA</i>	float;corrente ma.
<i>potencia_mW</i>	float;potencia mw.
<i>mediaVolts</i>	float;media volts.
<i>mediaAmperes</i>	float;media amperes.

6.3.2.2 inicialINA219()

```
void inicialINA219 ( )
```

inicia chip sensores de tensao e corrente ina219

Inicia INA219, sensor de tensao e corrente, para saber carga do sistema.

Date

10/12/2019

6.3.2.3 leINA219()

```
void leINA219 ( )
```

le sensores-entradas analogicas de tensao e corrente do ina219

Le ina 219.

Date

10/12/2019

6.3.3 Variable Documentation

6.3.3.1 accAmperes

```
float accAmperes = 0
```

acumulado de amperes

6.3.3.2 accVolts

```
float accVolts = 0
```

acumulado de volts

6.3.3.3 mediaVolts

```
float mediaVolts = 0
```

media de volts medidos/lidos

6.3.3.4 n_ina

```
int16_t n_ina = 0
```

rascunho de valor do ina219

6.4 Nextion-HMI-Display

rotinas-funcoes relacionadas ao hardware

Functions

- void [clarkProt_iniciarPopCallback](#) (void *ptr)

***** PAGINAS definidas no [mario_nextion.h](#) ***** `NexPage page0 = NexPage(0, 0, "page0");//menuPrinc-
Menu Principal NexPage page1 = NexPage(1, 0, "page1");//clarkProt-Protocolo CLARK NexPage page2 = Nex-
Page(2, 0, "page2");//teste01-pag de teste geral NexPage page3 = NexPage(3, 0, "page3");//rifePre-Protocolo R-
IFE pre- definido NexPage page4 = NexPage(4, 0, "page4");//zappicProt-Protocolo Zappicator NexPage page5
= NexPage(5, 0, "page5");//rifeETDFL-Protocolo Rife c/ETDFL 1017- 2018 NexPage page6 = NexPage(6, 0,
"page6");//pagFig-pag com figura grande NexPage page7 = NexPage(7, 0, "page7");//rifeManual- pag RIFE Man-
ual 6 seq de freq+tempo`
- void [rifePre_iniciarPopCallback](#) (void *ptr)

Callback, called when the rife pre qdo botao iniciar pop.
- void [zappicProt_iniciarPopCallback](#) (void *ptr)

Callback, called when the zappic prot qdo botao iniciar pop.
- void [rifeETDFL_iniciarPopCallback](#) (void *ptr)

Callback, called when the rife etdfl qdo botao iniciar pop.
- void [rifeManual_iniciarPopCallback](#) (void *ptr)

Callback, called when the rife manual qdo botao iniciar pop.
- void [clarkProt_cancelarPopCallback](#) (void *ptr)

Callback, called when the clark prot qdo botao cancelar pop.
- void [rifePre_cancelarPopCallback](#) (void *ptr)

Callback, called when the rife pre qdo botao cancelar pop.
- void [zappicProt_cancelarPopCallback](#) (void *ptr)

Callback, called when the zappic prot qdo botao cancelar pop.
- void [rifeETDFL_cancelarPopCallback](#) (void *ptr)

Callback, called when the rife etdfl qdo botao cancelar pop.
- void [rifeManual_cancelarPopCallback](#) (void *ptr)

Callback, called when the rife manual qdo botao cancelar pop.
- void [rifePre_bicho1PopCallback](#) (void *ptr)

===== RIFE Predefinido (bicho-1) botao
- void [rifePre_bicho2PopCallback](#) (void *ptr)

Callback, called when the rife pre bicho 2 botao pop.
- void [rifePre_bicho3PopCallback](#) (void *ptr)

Callback, called when the rife pre bicho 3 botao pop.
- void [preparaRifeBichoX](#) (int qualRife)

Prepara rife bicho (le tabela de frequencias)
- void [rifeETDFL_okPopCallback](#) (void *ptr)

RIFE(2)-teclado botao OK, coloca no display o nome e etapas.
- void [rifeETDFL_n1PopCallback](#) (void *ptr)

Callback, called when the rife etdfl botao tecla n 1 pop.
- void [rifeETDFL_n2PopCallback](#) (void *ptr)

Callback, called when the rife etdfl botao tecla n 2 pop.
- void [rifeETDFL_n3PopCallback](#) (void *ptr)

Callback, called when the rife etdfl botao tecla n 3 pop.
- void [rifeETDFL_n4PopCallback](#) (void *ptr)

Callback, called when the rife etdfl botao tecla n 4 pop.
- void [rifeETDFL_n5PopCallback](#) (void *ptr)

- Callback, called when the rife etdfl botao tecla n 5 pop.*
- void [rifeETDFL_n6PopCallback](#) (void *ptr)
 - Callback, called when the rife etdfl botao tecla n 6 pop.*
- void [rifeETDFL_n7PopCallback](#) (void *ptr)
 - Callback, called when the rife etdfl botao tecla n 7 pop.*
- void [rifeETDFL_n8PopCallback](#) (void *ptr)
 - Callback, called when the rife etdfl botao tecla n 8 pop.*
- void [rifeETDFL_n9PopCallback](#) (void *ptr)
 - Callback, called when the rife etdfl botao tecla n 9 pop.*
- void [rifeETDFL_n0PopCallback](#) (void *ptr)
 - Callback, called when the rife etdfl botao tecla n 0 pop.*
- void [rifeETDFL_nCPopCallback](#) (void *ptr)
 - Callback, called when the rife etdfl botao tecla n c pop (limpa/cancela)*
- void [rifeETDFL_numeroProtocoloPopCallback](#) (void *ptr)
 - Callback, called when the rife etdfl numero protocolo pop.*
- void [zappicProt_1KPopCallback](#) (void *ptr)
 - Callback, Zappicator de 1 KHz botao pop.*
- void [zappicProt_2K5PopCallback](#) (void *ptr)
 - Callback, called when the zappic prot 2.5 KHz botao pop.*
- void [zappicProt_428PopCallback](#) (void *ptr)
 - Callback, called when the zappic prot 428 Hz botao pop.*
- void [zappicProt_529PopCallback](#) (void *ptr)
 - Callback, called when the zappic prot 529 Hz botao pop.*
- void [rifeManual_n1PopCallback](#) (void *ptr)
 - Callback, rife manual botao n1.*
- void [rifeManual_n2PopCallback](#) (void *ptr)
 - Callback, called when the rife manual botao n 2 pop.*
- void [rifeManual_n3PopCallback](#) (void *ptr)
 - Callback, called when the rife manual botao n 3 pop.*
- void [rifeManual_n4PopCallback](#) (void *ptr)
 - Callback, called when the rife manual botao n 4 pop.*
- void [rifeManual_n5PopCallback](#) (void *ptr)
 - Callback, called when the rife manual botao n 5 pop.*
- void [rifeManual_n6PopCallback](#) (void *ptr)
 - Callback, called when the rife manual botao n 6 pop.*
- void [rifeManual_n7PopCallback](#) (void *ptr)
 - Callback, called when the rife manual botao n 7 pop.*
- void [rifeManual_n8PopCallback](#) (void *ptr)
 - Callback, called when the rife manual botao n 8 pop.*
- void [rifeManual_n9PopCallback](#) (void *ptr)
 - Callback, called when the rife manual botao n 9 pop.*
- void [rifeManual_n0PopCallback](#) (void *ptr)
 - Callback, called when the rife manual botao n 0 pop.*
- void [rifeManual_nCPopCallback](#) (void *ptr)
 - Callback, called when the rife manual botao n c pop (cancela/limpa)*
- void [rifeManual_freq01PopCallback](#) (void *ptr)
 - Callback, called when the rife manual le frequency 01 pop.*
- void [rifeManual_freq02PopCallback](#) (void *ptr)
 - Callback, called when the rife manual le frequency 02 pop.*
- void [rifeManual_freq03PopCallback](#) (void *ptr)
 - Callback, called when the rife manual le frequency 03 pop.*

- void `rifeManual_freq04PopCallback` (void *ptr)
Callback, called when the rife manual le frequency 04 pop.
- void `rifeManual_freq05PopCallback` (void *ptr)
Callback, called when the rife manual le frequency 05 pop.
- void `rifeManual_freq06PopCallback` (void *ptr)
Callback, called when the rife manual le frequency 06 pop.
- void `rifeManual_dura01PopCallback` (void *ptr)
Callback, called when the rife manual le duracao 01 pop.
- void `rifeManual_dura02PopCallback` (void *ptr)
Callback, called when the rife manual le duracao 02 pop.
- void `rifeManual_dura03PopCallback` (void *ptr)
Callback, called when the rife manual le duracao 03 pop.
- void `rifeManual_dura04PopCallback` (void *ptr)
Callback, called when the rife manual le duracao 04 pop.
- void `rifeManual_dura05PopCallback` (void *ptr)
Callback, called when the rife manual le duracao 05 pop.
- void `rifeManual_dura06PopCallback` (void *ptr)
Callback, called when the rife manual le duracao 06 pop.

Variables

- char `bufferNextion` [90]
buffer de comunicacao do-para NEXTION
- uint32_t `barraNextionRasc`
variavel da barra (p/incre-decre-mentar) do Nextion
- int `k`
int K do processo - contador
- int `tamComando`
tamanho do telegrama-comando Nextion
- char `bufferNextionRasc` [20]
buffer rascunho/buffer de tx-rx Nextion

6.4.1 Detailed Description

rotinas-funcoes relacionadas ao hardware

6.4.2 Function Documentation

6.4.2.1 `clarkProt_cancelarPopCallback()`

```
void clarkProt_cancelarPopCallback (
    void * ptr )
```

Callback, called when the clark prot qdo botao cancelar pop.

Date

10/12/2019

Parameters

<i>in, out</i>	<i>ptr</i>	If non-null, the pointer.
----------------	------------	---------------------------

6.4.2.2 clarkProt_iniciarPopCallback()

```
void clarkProt_iniciarPopCallback (
    void * ptr )
```

*****PAGINAS definidas no [mario_nextion.h](#) *****
 NexPage page0 = NexPage(0, 0, "page0");//menuPrinc-
 Menu Principal NexPage page1 = NexPage(1, 0, "page1"); //clarkProt-Protocolo CLARK NexPage page2 = Nex-
 Page(2, 0, "page2");//teste01-pag de teste geral NexPage page3 = NexPage(3, 0, "page3");//rifePre-Protocolo R-
 IFE pre- definido NexPage page4 = NexPage(4, 0, "page4");//zappicProt-Protocolo Zappicator NexPage page5
 = NexPage(5, 0, "page5");//rifeETDFL-Protocolo Rife c/ETDFL 1017- 2018 NexPage page6 = NexPage(6, 0,
 "page6");//pagFig-pag com figura grande NexPage page7 = NexPage(7, 0, "page7");//rifeManual- pag RIFE Manual
 6 seq de freq+tempo

Date

10/12/2019

Parameters

<i>in, out</i>	<i>ptr</i>	If non-null, the pointer.
----------------	------------	---------------------------

< inicia protocolo CLARK

6.4.2.3 preparaRifeBichoX()

```
void preparaRifeBichoX (
    int qualRife )
```

Prepara rife bicho (le tabela de frequencias)

Date

10/12/2019

Parameters

<i>qualRife</i>	The qual rife.
-----------------	----------------

6.4.2.4 rifeETDFL_cancelarPopCallback()

```
void rifeETDFL_cancelarPopCallback (
    void * ptr )
```

Callback, called when the rife etdfl qdo botao cancelar pop.

Date

10/12/2019

Parameters

in, out	ptr	If non-null, the pointer.
---------	-----	---------------------------

6.4.2.5 rifeETDFL_iniciarPopCallback()

```
void rifeETDFL_iniciarPopCallback (
    void * ptr )
```

Callback, called when the rife etdfl qdo botao iniciar pop.

Date

10/12/2019

Parameters

in, out	ptr	If non-null, the pointer.
---------	-----	---------------------------

< inicia protocolo RIFE (2)

6.4.2.6 rifeETDFL_n0PopCallback()

```
void rifeETDFL_n0PopCallback (
    void * ptr )
```

Callback, called when the rife etdfl botao tecla n 0 pop.

Date

10/12/2019

Parameters

in	ptr	If non-null, the pointer.
----	-----	---------------------------

6.4.2.7 rifeETDFL_n1PopCallback()

```
void rifeETDFL_n1PopCallback (  
    void * ptr )
```

Callback, called when the rife etdfl botao tecla n 1 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.8 rifeETDFL_n2PopCallback()

```
void rifeETDFL_n2PopCallback (  
    void * ptr )
```

Callback, called when the rife etdfl botao tecla n 2 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.9 rifeETDFL_n3PopCallback()

```
void rifeETDFL_n3PopCallback (  
    void * ptr )
```

Callback, called when the rife etdfl botao tecla n 3 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.10 rifeETDFL_n4PopCallback()

```
void rifeETDFL_n4PopCallback (  
    void * ptr )
```

Callback, called when the rife etdfl botao tecla n 4 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.11 rifeETDFL_n5PopCallback()

```
void rifeETDFL_n5PopCallback (  
    void * ptr )
```

Callback, called when the rife etdfl botao tecla n 5 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.12 rifeETDFL_n6PopCallback()

```
void rifeETDFL_n6PopCallback (  
    void * ptr )
```

Callback, called when the rife etdfl botao tecla n 6 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.13 rifeETDFL_n7PopCallback()

```
void rifeETDFL_n7PopCallback (  
    void * ptr )
```

Callback, called when the rife etdfl botao tecla n 7 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.14 rifeETDFL_n8PopCallback()

```
void rifeETDFL_n8PopCallback (  
    void * ptr )
```

Callback, called when the rife etdfl botao tecla n 8 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.15 rifeETDFL_n9PopCallback()

```
void rifeETDFL_n9PopCallback (  
    void * ptr )
```


Callback, called when the rife etdfl botao tecla n 9 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.16 rifeETDFl_nCPopCallback()

```
void rifeETDFl_nCPopCallback (  
    void * ptr )
```

Callback, called when the rife etdfl botao tecla n c pop (limpa/cancela)

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.17 rifeETDFl_numeroProtocoloPopCallback()

```
void rifeETDFl_numeroProtocoloPopCallback (  
    void * ptr )
```

Callback, called when the rife etdfl numero protocolo pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.18 rifeETDFL_okPopCallback()

```
void rifeETDFL_okPopCallback (  
    void * ptr )
```

RIFE(2)-teclado botao OK, coloca no display o nome e etapas.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.19 rifeManual_cancelarPopCallback()

```
void rifeManual_cancelarPopCallback (  
    void * ptr )
```

Callback, called when the rife manual qdo botao cancelar pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.20 rifeManual_dura01PopCallback()

```
void rifeManual_dura01PopCallback (  
    void * ptr )
```

Callback, called when the rife manual le duracao 01 pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.21 rifeManual_dura02PopCallback()

```
void rifeManual_dura02PopCallback (
    void * ptr )
```

Callback, called when the rife manual le duracao 02 pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.22 rifeManual_dura03PopCallback()

```
void rifeManual_dura03PopCallback (
    void * ptr )
```

Callback, called when the rife manual le duracao 03 pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.23 rifeManual_dura04PopCallback()

```
void rifeManual_dura04PopCallback (
    void * ptr )
```

Callback, called when the rife manual le duracao 04 pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.24 rifeManual_dura05PopCallback()

```
void rifeManual_dura05PopCallback (  
    void * ptr )
```

Callback, called when the rife manual le duracao 05 pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.25 rifeManual_dura06PopCallback()

```
void rifeManual_dura06PopCallback (  
    void * ptr )
```

Callback, called when the rife manual le duracao 06 pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.26 rifeManual_freq01PopCallback()

```
void rifeManual_freq01PopCallback (  
    void * ptr )
```

Callback, called when the rife manual le frequency 01 pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.27 rifeManual_freq02PopCallback()

```
void rifeManual_freq02PopCallback (  
    void * ptr )
```

Callback, called when the rife manual le frequency 02 pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.28 rifeManual_freq03PopCallback()

```
void rifeManual_freq03PopCallback (  
    void * ptr )
```

Callback, called when the rife manual le frequency 03 pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.29 rifeManual_freq04PopCallback()

```
void rifeManual_freq04PopCallback (  
    void * ptr )
```

Callback, called when the rife manual le frequency 04 pop.

Date

10/12/2019

Parameters

<i>in</i> , <i>out</i>	<i>ptr</i>	If non-null, the pointer.
------------------------	------------	---------------------------

6.4.2.30 rifeManual_freq05PopCallback()

```
void rifeManual_freq05PopCallback (  
    void * ptr )
```

Callback, called when the rife manual le frequency 05 pop.

Date

10/12/2019

Parameters

<i>in</i> , <i>out</i>	<i>ptr</i>	If non-null, the pointer.
------------------------	------------	---------------------------

6.4.2.31 rifeManual_freq06PopCallback()

```
void rifeManual_freq06PopCallback (  
    void * ptr )
```

Callback, called when the rife manual le frequency 06 pop.

Date

10/12/2019

Parameters

<i>in</i> , <i>out</i>	<i>ptr</i>	If non-null, the pointer.
------------------------	------------	---------------------------

6.4.2.32 rifeManual_iniciarPopCallback()

```
void rifeManual_iniciarPopCallback (
    void * ptr )
```

Callback, called when the rife manual qdo botao iniciar pop.

Date

10/12/2019

Parameters

in, out	ptr	If non-null, the pointer.
---------	-----	---------------------------

< inicia protocolo opManual

6.4.2.33 rifeManual_n0PopCallback()

```
void rifeManual_n0PopCallback (
    void * ptr )
```

Callback, called when the rife manual botao n 0 pop.

Date

10/12/2019

Parameters

in, out	ptr	If non-null, the pointer.
---------	-----	---------------------------

6.4.2.34 rifeManual_n1PopCallback()

```
void rifeManual_n1PopCallback (
    void * ptr )
```

Callback, rife manual botao n1.

Date

10/12/2019

Parameters

in	ptr	If non-null, the pointer.
----	-----	---------------------------

6.4.2.35 rifeManual_n2PopCallback()

```
void rifeManual_n2PopCallback (
    void * ptr )
```

Callback, called when the rife manual botao n 2 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.36 rifeManual_n3PopCallback()

```
void rifeManual_n3PopCallback (
    void * ptr )
```

Callback, called when the rife manual botao n 3 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.37 rifeManual_n4PopCallback()

```
void rifeManual_n4PopCallback (
    void * ptr )
```

Callback, called when the rife manual botao n 4 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.38 rifeManual_n5PopCallback()

```
void rifeManual_n5PopCallback (  
    void * ptr )
```

Callback, called when the rife manual botao n 5 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.39 rifeManual_n6PopCallback()

```
void rifeManual_n6PopCallback (  
    void * ptr )
```

Callback, called when the rife manual botao n 6 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.40 rifeManual_n7PopCallback()

```
void rifeManual_n7PopCallback (  
    void * ptr )
```

Callback, called when the rife manual botao n 7 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.41 rifeManual_n8PopCallback()

```
void rifeManual_n8PopCallback (  
    void * ptr )
```

Callback, called when the rife manual botao n 8 pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.42 rifeManual_n9PopCallback()

```
void rifeManual_n9PopCallback (  
    void * ptr )
```

Callback, called when the rife manual botao n 9 pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.43 rifeManual_nCPopCallback()

```
void rifeManual_nCPopCallback (  
    void * ptr )
```

Callback, called when the rife manual botao n c pop (cancela/limpa)

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.44 rifePre_bicho1PopCallback()

```
void rifePre_bicho1PopCallback (  
    void * ptr )
```

==== RIFE Predefindo (bicho-1) botao

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.45 rifePre_bicho2PopCallback()

```
void rifePre_bicho2PopCallback (  
    void * ptr )
```

Callback, called when the rife pre bicho 2 botao pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.46 rifePre_bicho3PopCallback()

```
void rifePre_bicho3PopCallback (  
    void * ptr )
```

Callback, called when the rife pre bicho 3 botao pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.47 rifePre_cancelarPopCallback()

```
void rifePre_cancelarPopCallback (  
    void * ptr )
```

Callback, called when the rife pre qdo botao cancelar pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.48 rifePre_iniciarPopCallback()

```
void rifePre_iniciarPopCallback (  
    void * ptr )
```

Callback, called when the rife pre qdo botao iniciar pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

< inicia protocolo RIFE (1)

6.4.2.49 zappicProt_1KPopCallback()

```
void zappicProt_1KPopCallback (  
    void * ptr )
```

Callback, Zappicator de 1 KHz botao pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.50 zappicProt_2K5PopCallback()

```
void zappicProt_2K5PopCallback (  
    void * ptr )
```

Callback, called when the zappic prot 2.5 KHz botao pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.51 zappicProt_428PopCallback()

```
void zappicProt_428PopCallback (  
    void * ptr )
```

Callback, called when the zappic prot 428 Hz botao pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.52 zappicProt_529PopCallback()

```
void zappicProt_529PopCallback (  
    void * ptr )
```

Callback, called when the zappic prot 529 Hz botao pop.

Date

10/12/2019

Parameters

in	<i>ptr</i>	If non-null, the pointer.
----	------------	---------------------------

6.4.2.53 zappicProt_cancelarPopCallback()

```
void zappicProt_cancelarPopCallback (  
    void * ptr )
```

Callback, called when the zappic prot qdo botao cancelar pop.

Date

10/12/2019

Parameters

in, out	<i>ptr</i>	If non-null, the pointer.
---------	------------	---------------------------

6.4.2.54 zappicProt_iniciarPopCallback()

```
void zappicProt_iniciarPopCallback (  
    void * ptr )
```

Callback, called when the zappic prot qdo botao iniciar pop.

Date

10/12/2019

Parameters

<i>in</i> , <i>out</i>	<i>ptr</i>	If non-null, the pointer.
------------------------	------------	---------------------------

< inicia protocolo ZAPPICATOR

Chapter 7

Class Documentation

7.1 Adafruit_INA219 Class Reference

Class that stores state and functions for interacting with INA219 current/power monitor IC.

```
#include <Adafruit_INA219.h>
```

Public Member Functions

- [Adafruit_INA219](#) (uint8_t addr=INA219_ADDRESS)
Instantiates a new INA219 class.
- void [begin](#) (void)
Setups the HW using the default Wire object.
- void [begin](#) (TwoWire *theWire)
Setups the HW (defaults to 32V and 2A for calibration values)
- void [setCalibration_32V_2A](#) (void)
Configures to INA219 to be able to measure up to 32V and 2A of current. Each unit of current corresponds to 100uA, and each unit of power corresponds to 2mW. Counter overflow occurs at 3.2A.
- void [setCalibration_32V_1A](#) (void)
Configures to INA219 to be able to measure up to 32V and 1A of current. Each unit of current corresponds to 40uA, and each unit of power corresponds to 800uW. Counter overflow occurs at 1.3A.
- void [setCalibration_16V_400mA](#) (void)
set device to alibration which uses the highest precision for current measurement (0.1mA), at the expense of only supporting 16V at 400mA max.
- float [getBusVoltage_V](#) (void)
Gets the shunt voltage in volts.
- float [getShuntVoltage_mV](#) (void)
Gets the shunt voltage in mV (so +/-327mV)
- float [getCurrent_mA](#) (void)
Gets the current value in mA, taking into account the config settings and current LSB.
- float [getPower_mW](#) (void)
Gets the power value in mW, taking into account the config settings and current LSB.

7.1.1 Detailed Description

Class that stores state and functions for interacting with INA219 current/power monitor IC.

7.1.2 Constructor & Destructor Documentation

7.1.2.1 Adafruit_INA219()

```
Adafruit_INA219::Adafruit_INA219 (
    uint8_t addr = INA219\_ADDRESS )
```

Instantiates a new INA219 class.

Parameters

<i>addr</i>	the I2C address the device can be found on. Default is 0x40
-------------	---

7.1.3 Member Function Documentation

7.1.3.1 begin()

```
void Adafruit_INA219::begin (
    TwoWire * theWire )
```

Setups the HW (defaults to 32V and 2A for calibration values)

Parameters

<i>theWire</i>	the TwoWire object to use
----------------	---------------------------

7.1.3.2 getBusVoltage_V()

```
float Adafruit_INA219::getBusVoltage_V (
    void )
```

Gets the shunt voltage in volts.

Returns

the bus voltage converted to volts

7.1.3.3 getCurrent_mA()

```
float Adafruit_INA219::getCurrent_mA (
    void )
```

Gets the current value in mA, taking into account the config settings and current LSB.

Returns

the current reading converted to milliamps

7.1.3.4 getPower_mW()

```
float Adafruit_INA219::getPower_mW (
    void )
```

Gets the power value in mW, taking into account the config settings and current LSB.

Returns

power reading converted to milliwatts

7.1.3.5 getShuntVoltage_mV()

```
float Adafruit_INA219::getShuntVoltage_mV (
    void )
```

Gets the shunt voltage in mV (so +-327mV)

Returns

the shunt voltage converted to millivolts

7.1.3.6 setCalibration_32V_1A()

```
void Adafruit_INA219::setCalibration_32V_1A (
    void )
```

Configures to INA219 to be able to measure up to 32V and 1A of current. Each unit of current corresponds to 40uA, and each unit of power corresponds to 800uW. Counter overflow occurs at 1.3A.

Note

These calculations assume a 0.1 ohm resistor is present

7.1.3.7 setCalibration_32V_2A()

```
void Adafruit_INA219::setCalibration_32V_2A (
    void )
```

Configures to INA219 to be able to measure up to 32V and 2A of current. Each unit of current corresponds to 100uA, and each unit of power corresponds to 2mW. Counter overflow occurs at 3.2A.

Note

These calculations assume a 0.1 ohm resistor is present

The documentation for this class was generated from the following files:

- C:/Users/mario/Documents/Arduino/ESP32_zapperV9.0/arquivos-dox/Adafruit_INA219/[Adafruit_INA219.h](#)
- C:/Users/mario/Documents/Arduino/ESP32_zapperV9.0/arquivos-dox/Adafruit_INA219/Adafruit_INA219.↔
cpp

Chapter 8

File Documentation

8.1 C:/Users/mario/Documents/Arduino/ESP32_zapperV9.0/arquivos-dox/Adafruit_INA219/Adafruit_INA219.h File Reference

```
#include "WProgram.h"
#include <Wire.h>
```

Classes

- class [Adafruit_INA219](#)
Class that stores state and functions for interacting with INA219 current/power monitor IC.

Macros

- #define [INA219_ADDRESS](#) (0x40)
default I2C address
- #define [INA219_READ](#) (0x01)
read
- #define [INA219_REG_CONFIG](#) (0x00)
config register address
- #define [INA219_CONFIG_RESET](#) (0x8000)
reset bit
- #define [INA219_CONFIG_BVOLTAGERANGE_MASK](#) (0x2000)
mask for bus voltage range
- #define [INA219_CONFIG_GAIN_MASK](#) (0x1800)
mask for gain bits
- #define [INA219_CONFIG_BADCRES_MASK](#) (0x0780)
mask for bus ADC resolution bits
- #define [INA219_CONFIG_SADCRES_MASK](#) (0x0078)
mask for shunt ADC resolution bits
- #define [INA219_CONFIG_MODE_MASK](#) (0x0007)
mask for operating mode bits

- `#define INA219_REG_SHUNTVOLTAGE (0x01)`
shunt voltage register
- `#define INA219_REG_BUSVOLTAGE (0x02)`
bus voltage register
- `#define INA219_REG_POWER (0x03)`
power register
- `#define INA219_REG_CURRENT (0x04)`
current register
- `#define INA219_REG_CALIBRATION (0x05)`
calibration register

Enumerations

- enum { `INA219_CONFIG_BVOLTAGERANGE_16V` = (0x0000), `INA219_CONFIG_BVOLTAGERANGE_32V` = (0x2000) }
bus voltage range values
- enum { `INA219_CONFIG_GAIN_1_40MV` = (0x0000), `INA219_CONFIG_GAIN_2_80MV` = (0x0800), `INA219_CONFIG_GAIN_4_160MV` = (0x1000), `INA219_CONFIG_GAIN_8_320MV` = (0x1800) }
values for gain bits
- enum { `INA219_CONFIG_BADCRES_9BIT` = (0x0000), `INA219_CONFIG_BADCRES_10BIT` = (0x0080), `INA219_CONFIG_BADCRES_11BIT` = (0x0100), `INA219_CONFIG_BADCRES_12BIT` = (0x0180) }
values for bus ADC resolution
- enum {
`INA219_CONFIG_SADCRES_9BIT_1S_84US` = (0x0000), `INA219_CONFIG_SADCRES_10BIT_1S_148US` = (0x0008), `INA219_CONFIG_SADCRES_11BIT_1S_276US` = (0x0010), `INA219_CONFIG_SADCRES_12BIT_1S_532US` = (0x0018),
`INA219_CONFIG_SADCRES_12BIT_2S_1060US` = (0x0048), `INA219_CONFIG_SADCRES_12BIT_4S_2130US` = (0x0050), `INA219_CONFIG_SADCRES_12BIT_8S_4260US` = (0x0058), `INA219_CONFIG_SADCRES_12BIT_16S_8510US` = (0x0060),
`INA219_CONFIG_SADCRES_12BIT_32S_17MS` = (0x0068), `INA219_CONFIG_SADCRES_12BIT_64S_34MS` = (0x0070), `INA219_CONFIG_SADCRES_12BIT_128S_69MS` = (0x0078) }
values for shunt ADC resolution
- enum {
`INA219_CONFIG_MODE_POWERDOWN` = (0x0000), `INA219_CONFIG_MODE_SVOLT_TRIGGERED` = (0x0001), `INA219_CONFIG_MODE_BVOLT_TRIGGERED` = (0x0002), `INA219_CONFIG_MODE_SANDBVOLT_TRIGGERED` = (0x0003),
`INA219_CONFIG_MODE_ADCOFF` = (0x0004), `INA219_CONFIG_MODE_SVOLT_CONTINUOUS` = (0x0005), `INA219_CONFIG_MODE_BVOLT_CONTINUOUS` = (0x0006), `INA219_CONFIG_MODE_SANDBVOLT_CONTINUOUS` = (0x0007) }
values for operating mode

8.1.1 Detailed Description

This is a library for the Adafruit INA219 breakout board —> <https://www.adafruit.com/products/904>

Adafruit invests time and resources providing this open source code, please support Adafruit and open-source hardware by purchasing products from Adafruit!

Written by Kevin "KTOWN" Townsend for Adafruit Industries.

BSD license, all text here must be included in any redistribution.

8.2 C:/Users/mario/Documents/Arduino/ESP32_zapperV9.0/arquivos-dox/Arduino.h File Reference

```
#include <stdbool.h>
#include <stdint.h>
#include <stdarg.h>
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <inttypes.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "esp32-hal.h"
#include "esp8266-compat.h"
#include "soc/gpio_reg.h"
#include "stdlib_noniso.h"
#include "binary.h"
#include "pins_arduino.h"
```

Macros

- **#define PI** 3.1415926535897932384626433832795
- **#define HALF_PI** 1.5707963267948966192313216916398
- **#define TWO_PI** 6.283185307179586476925286766559
- **#define DEG_TO_RAD** 0.017453292519943295769236907684886
- **#define RAD_TO_DEG** 57.295779513082320876798154814105
- **#define EULER** 2.718281828459045235360287471352
- **#define SERIAL** 0x0
- **#define DISPLAY** 0x1
- **#define LSBFIRST** 0
- **#define MSBFIRST** 1
- **#define RISING** 0x01
- **#define FALLING** 0x02
- **#define CHANGE** 0x03
- **#define ONLOW** 0x04
- **#define ONHIGH** 0x05
- **#define ONLOW_WE** 0x0C
- **#define ONHIGH_WE** 0x0D
- **#define DEFAULT** 1
- **#define EXTERNAL** 0
- **#define __STRINGIFY(a)** #a
- **#define constrain(amt, low, high)** ((amt)<(low)?(low):((amt)>(high)?(high):(amt)))
- **#define radians(deg)** ((deg)*DEG_TO_RAD)
- **#define degrees(rad)** ((rad)*RAD_TO_DEG)
- **#define sq(x)** ((x)*(x))
- **#define sei()**
- **#define cli()**
- **#define interrupts()** sei()
- **#define noInterrupts()** cli()

- `#define clockCyclesPerMicrosecond() ( F_CPU / 1000000L )`
- `#define clockCyclesToMicroseconds(a) ( (a) / clockCyclesPerMicrosecond() )`
- `#define microsecondsToClockCycles(a) ( (a) * clockCyclesPerMicrosecond() )`
- `#define lowByte(w) ((uint8_t) ((w) & 0xff))`
- `#define highByte(w) ((uint8_t) ((w) >> 8))`
- `#define bitRead(value, bit) (((value) >> (bit)) & 0x01)`
- `#define bitSet(value, bit) ((value) |= (1UL << (bit)))`
- `#define bitClear(value, bit) ((value) &= ~(1UL << (bit)))`
- `#define bitWrite(value, bit, bitvalue) (bitvalue ? bitSet(value, bit) : bitClear(value, bit))`
- `#define _NOP() do { __asm__ volatile ("nop"); } while (0)`
- `#define bit(b) (1UL << (b))`
- `#define _BV(b) (1UL << (b))`
- `#define digitalPinToPort(pin) (((pin)>31)?1:0)`
- `#define digitalPinToBitMask(pin) (1UL << (((pin)>31)?((pin)-32):(pin)))`
- `#define digitalPinToTimer(pin) (0)`
- `#define analogInPinToBit(P) (P)`
- `#define portOutputRegister(port) ((volatile uint32_t*)(port)?GPIO_OUT1_REG:GPIO_OUT_REG)`
- `#define portInputRegister(port) ((volatile uint32_t*)(port)?GPIO_IN1_REG:GPIO_IN_REG)`
- `#define portModeRegister(port) ((volatile uint32_t*)(port)?GPIO_ENABLE1_REG:GPIO_ENABLE_REG)`
- `#define NOT_A_PIN -1`
- `#define NOT_A_PORT -1`
- `#define NOT_AN_INTERRUPT -1`
- `#define NOT_ON_TIMER 0`
- `#define _min(a, b) ((a)<(b)?(a):(b))`
- `#define _max(a, b) ((a)>(b)?(a):(b))`

Typedefs

- `typedef bool boolean`
- `typedef uint8_t byte`
- `typedef unsigned int word`

Functions

- void `setup` (void)
inicializacao do zapper V9.0
- void `loop` (void)
Loops this object.
- long `random` (long, long)
- void `randomSeed` (unsigned long)
- long `map` (long, long, long, long, long)
- void `init` (void)
- void `initVariant` (void)
- void `initArduino` (void)
- unsigned long `pulseIn` (uint8_t pin, uint8_t state, unsigned long timeout)
- unsigned long `pulseInLong` (uint8_t pin, uint8_t state, unsigned long timeout)
- uint8_t `shiftIn` (uint8_t dataPin, uint8_t clockPin, uint8_t bitOrder)
- void `shiftOut` (uint8_t dataPin, uint8_t clockPin, uint8_t bitOrder, uint8_t val)

8.2.1 Detailed Description

este eader "Arduino" alterado pela Espressif para funcionar com o ESP32 e FreeRTOS

Author

espressif - mltf

Date

2019/10/02

8.3 C:/Users/mario/Documents/Arduino/ESP32_zapperV9.0/arquivos-dox/mario_nextion.h File Reference

Functions

- void **initObjNextion** ()
- void **clarkProt_iniciarPopCallback** (void *ptr)

***** PAGINAS definidas no [mario_nextion.h](#) ***** *NexPage page0 = NexPage(0, 0, "page0");//menuPrinc-↵
Menu Principal NexPage page1 = NexPage(1, 0, "page1");//clarkProt-Protocolo CLARK NexPage page2 = Nex↵
Page(2, 0, "page2");//teste01-pag de teste geral NexPage page3 = NexPage(3, 0, "page3");//rifePre-Protocolo R↵
IFE pre- definido NexPage page4 = NexPage(4, 0, "page4");//zappicProt-Protocolo Zappicator NexPage page5
= NexPage(5, 0, "page5");//rifeETDFL-Protocolo Rife c/ETDFL 1017- 2018 NexPage page6 = NexPage(6, 0,
"page6");//pagFig-pag com figura grande NexPage page7 = NexPage(7, 0, "page7");//rifeManual- pag RIFE Man-
ual 6 seq de freq+tempo*
- void **rifePre_iniciarPopCallback** (void *ptr)

Callback, called when the rife pre qdo botao iniciar pop.
- void **zappicProt_iniciarPopCallback** (void *ptr)

Callback, called when the zappic prot qdo botao iniciar pop.
- void **rifeETDFL_iniciarPopCallback** (void *ptr)

Callback, called when the rife etdfl qdo botao iniciar pop.
- void **rifeManual_iniciarPopCallback** (void *ptr)

Callback, called when the rife manual qdo botao iniciar pop.
- void **clarkProt_cancelarPopCallback** (void *ptr)

Callback, called when the clark prot qdo botao cancelar pop.
- void **rifePre_cancelarPopCallback** (void *ptr)

Callback, called when the rife pre qdo botao cancelar pop.
- void **zappicProt_cancelarPopCallback** (void *ptr)

Callback, called when the zappic prot qdo botao cancelar pop.
- void **rifeETDFL_cancelarPopCallback** (void *ptr)

Callback, called when the rife etdfl qdo botao cancelar pop.
- void **rifeManual_cancelarPopCallback** (void *ptr)

Callback, called when the rife manual qdo botao cancelar pop.
- void **rifePre_bicho1PopCallback** (void *ptr)

==== RIFE Predefinido (bicho-1) botao
- void **rifePre_bicho2PopCallback** (void *ptr)

Callback, called when the rife pre bicho 2 botao pop.
- void **rifePre_bicho3PopCallback** (void *ptr)

- Callback, called when the rife pre bicho 3 botao pop.*
- void [zappicProt_1KPopCallback](#) (void *ptr)
- Callback, Zappicator de 1 KHz botao pop.*
- void [zappicProt_2K5PopCallback](#) (void *ptr)
- Callback, called when the zappic prot 2.5 KHz botao pop.*
- void [zappicProt_428PopCallback](#) (void *ptr)
- Callback, called when the zappic prot 428 Hz botao pop.*
- void [zappicProt_529PopCallback](#) (void *ptr)
- Callback, called when the zappic prot 529 Hz botao pop.*
- void [rifeETDFL_n0PopCallback](#) (void *ptr)
- Callback, called when the rife etdfl botao tecla n 0 pop.*
- void [rifeETDFL_n1PopCallback](#) (void *ptr)
- Callback, called when the rife etdfl botao tecla n 1 pop.*
- void [rifeETDFL_n2PopCallback](#) (void *ptr)
- Callback, called when the rife etdfl botao tecla n 2 pop.*
- void [rifeETDFL_n3PopCallback](#) (void *ptr)
- Callback, called when the rife etdfl botao tecla n 3 pop.*
- void [rifeETDFL_n4PopCallback](#) (void *ptr)
- Callback, called when the rife etdfl botao tecla n 4 pop.*
- void [rifeETDFL_n5PopCallback](#) (void *ptr)
- Callback, called when the rife etdfl botao tecla n 5 pop.*
- void [rifeETDFL_n6PopCallback](#) (void *ptr)
- Callback, called when the rife etdfl botao tecla n 6 pop.*
- void [rifeETDFL_n7PopCallback](#) (void *ptr)
- Callback, called when the rife etdfl botao tecla n 7 pop.*
- void [rifeETDFL_n8PopCallback](#) (void *ptr)
- Callback, called when the rife etdfl botao tecla n 8 pop.*
- void [rifeETDFL_n9PopCallback](#) (void *ptr)
- Callback, called when the rife etdfl botao tecla n 9 pop.*
- void [rifeETDFL_nCPopCallback](#) (void *ptr)
- Callback, called when the rife etdfl botao tecla n c pop (limpa/cancela)*
- void [rifeETDFL_okPopCallback](#) (void *ptr)
- RIFE(2)-teclado botao OK, coloca no display o nome e etapas.*
- void [rifeETDFL_numeroProtocoloPopCallback](#) (void *ptr)
- Callback, called when the rife etdfl numero protocolo pop.*
- void [rifeManual_n0PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 0 pop.*
- void [rifeManual_n1PopCallback](#) (void *ptr)
- Callback, rife manual botao n1.*
- void [rifeManual_n2PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 2 pop.*
- void [rifeManual_n3PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 3 pop.*
- void [rifeManual_n4PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 4 pop.*
- void [rifeManual_n5PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 5 pop.*
- void [rifeManual_n6PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 6 pop.*
- void [rifeManual_n7PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 7 pop.*

- void [rifeManual_n8PopCallback](#) (void *ptr)
Callback, called when the rife manual botao n 8 pop.
- void [rifeManual_n9PopCallback](#) (void *ptr)
Callback, called when the rife manual botao n 9 pop.
- void [rifeManual_nCPopCallback](#) (void *ptr)
Callback, called when the rife manual botao n c pop (cancela/limpa)
- void [rifeManual_dura01PopCallback](#) (void *ptr)
Callback, called when the rife manual le duracao 01 pop.
- void [rifeManual_dura02PopCallback](#) (void *ptr)
Callback, called when the rife manual le duracao 02 pop.
- void [rifeManual_dura03PopCallback](#) (void *ptr)
Callback, called when the rife manual le duracao 03 pop.
- void [rifeManual_dura04PopCallback](#) (void *ptr)
Callback, called when the rife manual le duracao 04 pop.
- void [rifeManual_dura05PopCallback](#) (void *ptr)
Callback, called when the rife manual le duracao 05 pop.
- void [rifeManual_dura06PopCallback](#) (void *ptr)
Callback, called when the rife manual le duracao 06 pop.
- void [rifeManual_freq01PopCallback](#) (void *ptr)
Callback, called when the rife manual le frequency 01 pop.
- void [rifeManual_freq02PopCallback](#) (void *ptr)
Callback, called when the rife manual le frequency 02 pop.
- void [rifeManual_freq03PopCallback](#) (void *ptr)
Callback, called when the rife manual le frequency 03 pop.
- void [rifeManual_freq04PopCallback](#) (void *ptr)
Callback, called when the rife manual le frequency 04 pop.
- void [rifeManual_freq05PopCallback](#) (void *ptr)
Callback, called when the rife manual le frequency 05 pop.
- void [rifeManual_freq06PopCallback](#) (void *ptr)
Callback, called when the rife manual le frequency 06 pop.

Variables

- NexPage **page0** = NexPage(0, 0, "page0")
- NexPage **page1** = NexPage(1, 0, "page1")
- NexPage **page2** = NexPage(2, 0, "page2")
- NexPage **page3** = NexPage(3, 0, "page3")
- NexPage **page4** = NexPage(4, 0, "page4")
- NexPage **page5** = NexPage(5, 0, "page5")
- NexPage **page6** = NexPage(6, 0, "page6")
- NexPage **page7** = NexPage(7, 0, "page7")
- NexVariable **pg0emPrograma** = NexVariable(0, 9, "page0.emPrograma")
- NexButton **clarkProt_iniciar** = NexButton(1, 3, "page1.b2")
- NexButton **clarkProt_cancelar** = NexButton(1, 5, "page1.b4")
- NexProgressBar **clarkProt_barra** = NexProgressBar(1, 2, "page1.j0")
- NexButton **rifePre_bicho1** = NexButton(3, 5, "page3.b1")
- NexButton **rifePre_bicho2** = NexButton(3, 6, "page3.b2")
- NexButton **rifePre_bicho3** = NexButton(3, 7, "page3.b3")
- NexButton **rifePre_iniciar** = NexButton(3, 8, "page3.b6")
- NexButton **rifePre_cancelar** = NexButton(3, 9, "page3.b7")
- NexProgressBar **rifePre_barra** = NexProgressBar(3, 3, "page3.j0")

- `NexText rifePre_frequencia = NexText(3, 12, "page3.t4")`
- `NexText rifePre_tempo = NexText(3, 13, "page3.t5")`
- `NexText rifePre_etapa = NexText(3, 14, "page3.t6")`
- `NexText rifePre_nomeProtocolo = NexText(3, 11, "page3.t3")`
- `NexButton zappicProt_iniciar = NexButton(4, 4, "page4.b1")`
- `NexButton zappicProt_cancelar = NexButton(4, 5, "page4.b2")`
- `NexProgressBar zappicProt_barra = NexProgressBar(4, 3, "page4.j0")`
- `NexNumber zappicProt_duracao = NexNumber(4, 16, "page4.n0")`
- `NexRadio zappicProt_1K = NexRadio(4, 6, "page4.r0")`
- `NexRadio zappicProt_2K5 = NexRadio(4, 7, "page4.r1")`
- `NexRadio zappicProt_428 = NexRadio(4, 8, "page4.r2")`
- `NexRadio zappicProt_529 = NexRadio(4, 9, "page4.r3")`
- `NexSlider zappicProt_slicer = NexSlider(4,15,"page4.h0")`
- `NexButton rifeETDFL_ok = NexButton(5, 4, "page5.b0")`
- `NexButton rifeETDFL_iniciar = NexButton(5, 18, "page5.b12")`
- `NexButton rifeETDFL_cancelar = NexButton(5, 19, "page5.b13")`
- `NexButton rifeETDFL_n1 = NexButton(5, 6, "page5.b1")`
- `NexButton rifeETDFL_n2 = NexButton(5, 8, "page5.b2")`
- `NexButton rifeETDFL_n3 = NexButton(5, 9, "page5.b3")`
- `NexButton rifeETDFL_n4 = NexButton(5, 10, "page5.b4")`
- `NexButton rifeETDFL_n5 = NexButton(5, 11, "page5.b5")`
- `NexButton rifeETDFL_n6 = NexButton(5, 12, "page5.b6")`
- `NexButton rifeETDFL_n7 = NexButton(5, 13, "page5.b7")`
- `NexButton rifeETDFL_n8 = NexButton(5, 14, "page5.b8")`
- `NexButton rifeETDFL_n9 = NexButton(5, 15, "page5.b9")`
- `NexButton rifeETDFL_n0 = NexButton(5, 16, "page5.b10")`
- `NexButton rifeETDFL_nC = NexButton(5, 21, "page5.b14")`
- `NexProgressBar rifeETDFL_barra = NexProgressBar(5, 5, "page5.j0")`
- `NexNumber rifeETDFL_numeroProtocolo = NexNumber(5, 2, "page5.n1")`
- `NexNumber rifeETDFL_etapa = NexNumber(5, 22, "page5.n0")`
- `NexText rifeETDFL_nomeProtocolo = NexText(5, 20, "page5.t3")`
- `NexText rifeETDFL_frequencia = NexText(5, 24, "page5.t5")`
- `NexButton rifeManual_iniciar = NexButton(7,29, "page7.b0")`
- `NexButton rifeManual_cancelar = NexButton(7,30, "page7.b11")`
- `NexText rifeManual_numero = NexText(7,18, "page7.t3")`
- `NexButton rifeManual_n1 = NexButton(7,17, "page7.b1")`
- `NexButton rifeManual_n2 = NexButton(7,19, "page7.b2")`
- `NexButton rifeManual_n3 = NexButton(7,20, "page7.b3")`
- `NexButton rifeManual_n4 = NexButton(7,21, "page7.b4")`
- `NexButton rifeManual_n5 = NexButton(7,22, "page7.b5")`
- `NexButton rifeManual_n6 = NexButton(7,23, "page7.b6")`
- `NexButton rifeManual_n7 = NexButton(7,24, "page7.b7")`
- `NexButton rifeManual_n8 = NexButton(7,25, "page7.b8")`
- `NexButton rifeManual_n9 = NexButton(7,26, "page7.b9")`
- `NexButton rifeManual_n0 = NexButton(7,27, "page7.b10")`
- `NexButton rifeManual_nC = NexButton(7, 28, "page7.b14")`
- `NexProgressBar rifeManual_barra = NexProgressBar(7,31, "page7.j0")`
- `NexNumber rifeManual_freq01 = NexNumber(7,5, "page7.n0")`
- `NexNumber rifeManual_freq02 = NexNumber(7,7, "page7.n2")`
- `NexNumber rifeManual_freq03 = NexNumber(7,9, "page7.n4")`
- `NexNumber rifeManual_freq04 = NexNumber(7,11, "page7.n6")`
- `NexNumber rifeManual_freq05 = NexNumber(7,13, "page7.n8")`
- `NexNumber rifeManual_freq06 = NexNumber(7,15, "page7.n10")`
- `NexNumber rifeManual_dura01 = NexNumber(7,6, "page7.n1")`
- `NexNumber rifeManual_dura02 = NexNumber(7,8, "page7.n3")`

- NexNumber **riFeManual_dura03** = NexNumber(7,10, "page7.n5")
- NexNumber **riFeManual_dura04** = NexNumber(7,12, "page7.n7")
- NexNumber **riFeManual_dura05** = NexNumber(7,14, "page7.n9")
- NexNumber **riFeManual_dura06** = NexNumber(7,16, "page7.n11")
- NexTouch * **nex_listen_list** []
- const unsigned long **nxtWHITE** =65535
- const unsigned long **nxtBLACK** =65535
- const unsigned long **nxtBLUE** =31
- const unsigned long **nxtBROWN** =48192
- const unsigned long **nxtGREEN** =2016
- const unsigned long **nxtYELLOW** =65504
- const unsigned long **nxtRED** =63488
- const unsigned long **nxtGRAY** =33840

8.3.1 Detailed Description

minhas definicoes de telas e objetos do nextion hmi.

Author

MLTF

Date

2018/05/02

8.3.2 Variable Documentation

8.3.2.1 nex_listen_list

NexTouch* nex_listen_list[]

Initial value:

```
=
{
    &page0, &page1, &page2, &page3, &page4, &page5, &page6, &page7,
    &clarkProt_iniciar,&clarkProt_cancelar,
    &riFePre_bicho1,    &riFePre_bicho2,    &riFePre_bicho3,    &riFePre_iniciar,&riFePre_cancelar,
    &zappicProt_iniciar,&zappicProt_cancelar,
    &zappicProt_1K,&zappicProt_2K5,&zappicProt_428,&zappicProt_529,
    &riFeETDFL_iniciar,&riFeETDFL_cancelar,&riFeETDFL_ok,
    &riFeETDFL_n1,&riFeETDFL_n2,&riFeETDFL_n3,&riFeETDFL_n4,&riFeETDFL_n5,
    &riFeETDFL_n6,&riFeETDFL_n7,&riFeETDFL_n8,&riFeETDFL_n9,&riFeETDFL_n0,&riFeETDFL_nC,
    &riFeETDFL_numeroProtocolo,
    &riFeManual_iniciar,&riFeManual_cancelar,
    &riFeManual_n1,&riFeManual_n2,&riFeManual_n3,&riFeManual_n4,&riFeManual_n5,
    &riFeManual_n6,&riFeManual_n7,&riFeManual_n8,&riFeManual_n9,&riFeManual_n0,&riFeManual_nC,
    &riFeManual_freq01, &riFeManual_freq02, &riFeManual_freq03,
    &riFeManual_freq04, &riFeManual_freq05, &riFeManual_freq06,
    &riFeManual_dura01,&riFeManual_dura02, &riFeManual_dura03,
    &riFeManual_dura04, &riFeManual_dura05,&riFeManual_dura06,
    NULL
}
```

8.4 C:/Users/mario/Documents/Arduino/ESP32_zapperV9.0/arquivos-dox/Nextion.h File Reference

```
#include "NexConfig.h"
#include "NexTouch.h"
#include "NexHardware.h"
#include "NexButton.h"
#include "NexCrop.h"
#include "NexGauge.h"
#include "NexHotspot.h"
#include "NexPage.h"
#include "NexPicture.h"
#include "NexProgressBar.h"
#include "NexSlider.h"
#include "NexText.h"
#include "NexWaveform.h"
#include "NexTimer.h"
#include "NexNumber.h"
#include "NexDualStateButton.h"
#include "NexVariable.h"
#include "NexCheckbox.h"
#include "NexRadio.h"
#include "NexScrolltext.h"
```

8.4.1 Detailed Description

The header file including all other header files provided by this library.

Every example sketch should include this file.

Author

Wu Pengfei (email: pengfei.wu@itead.cc)

Date

2015/8/12

Copyright

Copyright (C) 2014-2015 ITEAD Intelligent Systems Co., Ltd.

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

8.5 C:/Users/mario/Documents/Arduino/ESP32_zapperV9.0/ESP32_zapperV9.0.ino File Reference

Modulo Esp32-zapper v 9.0 funcoes de tratamento do PWM para zapper, clark, rifle, zappicator. funcoes da tela HMI - Nextion. funcoes de leitura do INA219 - sensor tensao e corrente.

```
#include <Wire.h>
#include <Adafruit_INA219.h>
#include <Arduino.h>
#include <Nextion.h>
#include <tabelas_rife-esp32.h>
#include <mario_nextion.h>
```

Functions

- void **daBeep** (int tempoBip)
 - *****DEFINICOES/PROTOTOPOS*****
- void **avisaXseg** (int tempoBip, int numeroBips)
 - ***** AVISO DE N BEEPS POR X SEG ***** 1 segundo entre bips (1000 ms) ***cuidado com rotina para nao travar processamento
- void **tickRelogio** ()
 - relogio feito por software no loop - nao vai usar
- void **iniciaDebug** ()
 - envia msg ao monitor inicio do debug
- void **dbgMostraDados** (String mensagem, int valor)
 - mostra msg e dados - monitor
- void **limpaStatus** (int protocolo)
 - limpa todos flags do protocolo em execucao
- void **tempoMarcaInicio** ()
 - guarda hora que comecou protocolo
- unsigned long **tempoLeAgora** ()
 - retorna false se estourou tempo
- void **iniciaPWMs** ()
 - inicia ondas quadradas dos protocolos-tratamentos
- void **terminaPrograma** (int protocolo)
 - termina protocolo-tratamento
- void **ligaPWM** (float pwmFreq)
 - liga pwm
- void **desligaPWM** ()
 - desliga pwm
- void **terminaProgramaZ** (int protocolo)
 - termina protocolo tipo zappicator
- void **ligaPWMZ** (float entraFreq)
 - liga pwm do tipo zappicator
- void **desligaPWMZ** ()
 - termina pwm do tipo zappicator
- void **inicialNA219** ()
 - inicia chip sensores de tensao e corrente ina219

- void [leINA219](#) ()
le sensores-entradas analogicas de tensao e corrente do ina219
- void [dbgMostralna219](#) (float voltagemCircuito, float voltagemShunt, float voltagemEntrada, float corrente_↔ mA, float potencia_mW, float [mediaVolts](#), float [mediaAmperes](#))
Debug mostra valores ina 219 (mA, mV, mW, media Volt, media A)
- void [comecaClark](#) ()
Inicia protocolo CLARK.
- void [proximoParClark](#) ()
Proximo par clark.
- void [clark_7min](#) ()
Controle de tempo (7 min) - CLARK.
- void [clark_20min](#) ()
Controle de tempo (20 min) - Clark.
- void [avancaBarraClark](#) (int incremento)
Avanca barra clark.
- void [comecaZappic](#) ()
comeca ZAPPICATOR
- void [avancaBarraZapp](#) (float tempoDivisor)
Avanca barra zapp.
- void [controleZappic](#) ()
Controle do zappicator.
- void [preparaRifeBichoX](#) (int [qualRife](#))
Prepara rife bicho (le tabela de frequencias)
- void [comecaRifePre](#) (int [qualRife](#))
Comeca RIFE PRE.
- void [proximoRifePre](#) ()
Proximo rife pre.
- void [avancaBarraRifePre](#) (float tempoDivisor)
ALGO ESTRANHO COM ESTA AVANCA BARRAS DO RIFE.
- void [controleRifePre](#) ()
Controle rife pre.
- void [comecaRifeETDFL](#) (int [qualRife](#))
RIFE ETDFL.
- void [proximoRifeETDFL](#) ()
Proximo rife etdfl.
- void [avancaBarraRifeETDFL](#) (float tempoDivisor)
Avanca barra rife etdfl.
- void [controleRifeETDFL](#) ()
Controle rife etdfl.
- void [comecaRifeManual](#) ()
*Comeca RIFE MANUAL vai ler frequencias e tempos; numero lido - getvalue - usa little endian order 0x01, 0x02, 0x03,0x04 (4 bytes na msg de retorno) 0x01+(0x02*256)+(0x03*65536)+(0x04*16777216)*
- void [proximoRifeManual](#) ()
Proximo rife manual.
- void [avancaBarraRifeManual](#) (float tempoDivisor)
Avanca barra rife manual.
- void [controleRifeManual](#) ()
Controle rife manual.
- void [setup](#) ()
inicializacao do zapper V9.0
- void [loop](#) ()

Loops this object.

- void [clarkProt_iniciarPopCallback](#) (void *ptr)

***** PAGINAS definidas no [mario_nextion.h](#) ***** NexPage page0 = NexPage(0, 0, "page0");//menuPrinc-
Menu Principal NexPage page1 = NexPage(1, 0, "page1");//clarkProt-Protocolo CLARK NexPage page2 = Nex-
Page(2, 0, "page2");//teste01-pag de teste geral NexPage page3 = NexPage(3, 0, "page3");//rifePre-Protocolo R-
IFE pre- definido NexPage page4 = NexPage(4, 0, "page4");//zappicProt-Protocolo Zappicator NexPage page5
= NexPage(5, 0, "page5");//rifeETDFL-Protocolo Rife c/ETDFL 1017- 2018 NexPage page6 = NexPage(6, 0,
"page6");//pagFig-pag com figura grande NexPage page7 = NexPage(7, 0, "page7");//rifeManual- pag RIFE Man-
ual 6 seq de freq+tempo
- void [rifePre_iniciarPopCallback](#) (void *ptr)

Callback, called when the rife pre qdo botao iniciar pop.
- void [zappicProt_iniciarPopCallback](#) (void *ptr)

Callback, called when the zappic prot qdo botao iniciar pop.
- void [rifeETDFL_iniciarPopCallback](#) (void *ptr)

Callback, called when the rife etdfl qdo botao iniciar pop.
- void [rifeManual_iniciarPopCallback](#) (void *ptr)

Callback, called when the rife manual qdo botao iniciar pop.
- void [clarkProt_cancelarPopCallback](#) (void *ptr)

Callback, called when the clark prot qdo botao cancelar pop.
- void [rifePre_cancelarPopCallback](#) (void *ptr)

Callback, called when the rife pre qdo botao cancelar pop.
- void [zappicProt_cancelarPopCallback](#) (void *ptr)

Callback, called when the zappic prot qdo botao cancelar pop.
- void [rifeETDFL_cancelarPopCallback](#) (void *ptr)

Callback, called when the rife etdfl qdo botao cancelar pop.
- void [rifeManual_cancelarPopCallback](#) (void *ptr)

Callback, called when the rife manual qdo botao cancelar pop.
- void [rifePre_bicho1PopCallback](#) (void *ptr)

==== RIFE Predefinido (bicho-1) botao
- void [rifePre_bicho2PopCallback](#) (void *ptr)

Callback, called when the rife pre bicho 2 botao pop.
- void [rifePre_bicho3PopCallback](#) (void *ptr)

Callback, called when the rife pre bicho 3 botao pop.
- void [rifeETDFL_okPopCallback](#) (void *ptr)

RIFE(2)-teclado botao OK, coloca no display o nome e etapas.
- void [rifeETDFL_n1PopCallback](#) (void *ptr)

Callback, called when the rife etdfl botao tecla n 1 pop.
- void [rifeETDFL_n2PopCallback](#) (void *ptr)

Callback, called when the rife etdfl botao tecla n 2 pop.
- void [rifeETDFL_n3PopCallback](#) (void *ptr)

Callback, called when the rife etdfl botao tecla n 3 pop.
- void [rifeETDFL_n4PopCallback](#) (void *ptr)

Callback, called when the rife etdfl botao tecla n 4 pop.
- void [rifeETDFL_n5PopCallback](#) (void *ptr)

Callback, called when the rife etdfl botao tecla n 5 pop.
- void [rifeETDFL_n6PopCallback](#) (void *ptr)

Callback, called when the rife etdfl botao tecla n 6 pop.
- void [rifeETDFL_n7PopCallback](#) (void *ptr)

Callback, called when the rife etdfl botao tecla n 7 pop.
- void [rifeETDFL_n8PopCallback](#) (void *ptr)

Callback, called when the rife etdfl botao tecla n 8 pop.
- void [rifeETDFL_n9PopCallback](#) (void *ptr)

- Callback, called when the rife etdfl botao tecla n 9 pop.*
- void [rifeETDFL_n0PopCallback](#) (void *ptr)
- Callback, called when the rife etdfl botao tecla n 0 pop.*
- void [rifeETDFL_nCPopCallback](#) (void *ptr)
- Callback, called when the rife etdfl botao tecla n c pop (limpa/cancela)*
- void [rifeETDFL_numeroProtocoloPopCallback](#) (void *ptr)
- Callback, called when the rife etdfl numero protocolo pop.*
- void [zappicProt_1KPopCallback](#) (void *ptr)
- Callback, Zappicator de 1 KHz botao pop.*
- void [zappicProt_2K5PopCallback](#) (void *ptr)
- Callback, called when the zappic prot 2.5 KHz botao pop.*
- void [zappicProt_428PopCallback](#) (void *ptr)
- Callback, called when the zappic prot 428 Hz botao pop.*
- void [zappicProt_529PopCallback](#) (void *ptr)
- Callback, called when the zappic prot 529 Hz botao pop.*
- void [rifeManual_n1PopCallback](#) (void *ptr)
- Callback, rife manual botao n1.*
- void [rifeManual_n2PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 2 pop.*
- void [rifeManual_n3PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 3 pop.*
- void [rifeManual_n4PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 4 pop.*
- void [rifeManual_n5PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 5 pop.*
- void [rifeManual_n6PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 6 pop.*
- void [rifeManual_n7PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 7 pop.*
- void [rifeManual_n8PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 8 pop.*
- void [rifeManual_n9PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 9 pop.*
- void [rifeManual_n0PopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n 0 pop.*
- void [rifeManual_nCPopCallback](#) (void *ptr)
- Callback, called when the rife manual botao n c pop (cancela/limpa)*
- void [rifeManual_freq01PopCallback](#) (void *ptr)
- Callback, called when the rife manual le frequency 01 pop.*
- void [rifeManual_freq02PopCallback](#) (void *ptr)
- Callback, called when the rife manual le frequency 02 pop.*
- void [rifeManual_freq03PopCallback](#) (void *ptr)
- Callback, called when the rife manual le frequency 03 pop.*
- void [rifeManual_freq04PopCallback](#) (void *ptr)
- Callback, called when the rife manual le frequency 04 pop.*
- void [rifeManual_freq05PopCallback](#) (void *ptr)
- Callback, called when the rife manual le frequency 05 pop.*
- void [rifeManual_freq06PopCallback](#) (void *ptr)
- Callback, called when the rife manual le frequency 06 pop.*
- void [rifeManual_dura01PopCallback](#) (void *ptr)
- Callback, called when the rife manual le duracao 01 pop.*

- void [rifeManual_dura02PopCallback](#) (void *ptr)
Callback, called when the rife manual le duracao 02 pop.
- void [rifeManual_dura03PopCallback](#) (void *ptr)
Callback, called when the rife manual le duracao 03 pop.
- void [rifeManual_dura04PopCallback](#) (void *ptr)
Callback, called when the rife manual le duracao 04 pop.
- void [rifeManual_dura05PopCallback](#) (void *ptr)
Callback, called when the rife manual le duracao 05 pop.
- void [rifeManual_dura06PopCallback](#) (void *ptr)
Callback, called when the rife manual le duracao 06 pop.

Variables

- bool [DEBUGANDO](#) = true
Variavel Global - boolean true indica que debug habilitado.
- [Adafruit_INA219](#) [ina219](#)
cria instancia do INA219 - [Adafruit_INA219](#) [ina219](#)
- unsigned long [tempoIna219Rasc](#) = 0
unsigned long, rascunho de tempo-timer do Ina219
- int16_t [n_ina](#) = 0
rascunho de valor do ina219
- float [mediaVolts](#) = 0
media de volts medidos/lidos
- float [mediaAmperes](#) = 0
media de amperes medidos/lidos
- float [accVolts](#) = 0
acumulado de volts
- float [accAmperes](#) = 0
acumulado de amperes
- bool [paraLeitura](#) = false
para temporariamente leitura
- int [buffer_itens](#) = 50
numero de itens (tamanho) do buffer
- int [buffer_v](#) [50]
o [buffer_v](#)[50] buffer de 50 posicoes
- unsigned int [segundos](#)
para relógio, timer e temporizadores
- unsigned int [minutos](#)
para relógio, timer e temporizadores
- unsigned int [horas](#)
para relógio, timer e temporizadores
- unsigned int [n7](#)
para relógio, timer e temporizadores
- unsigned long [tempoAgora](#) = 0
tempo agora
- unsigned long [tempoAntes](#) = 0
tempo antes
- unsigned long [tempoInicial](#)
rascunho de tempo no momento que foi ligado/desligado
- unsigned long [tempoSegundos](#)

- tempo em segundos*
- char `bufferNextion` [90]
 - buffer de comunicacao do-para NEXTION*
- uint32_t `barraNextionRasc`
 - variavel da barra (p/incre-decre-mentar) do Nextion*
- int `k`
 - int K do processo - contador*
- int `tamComando`
 - tamanho do telegrama-comando Nextion*
- char `bufferNextionRasc` [20]
 - buffer rascunho/buffer de tx-rx Nextion*
- bool `emPrograma` = false
 - ***** flag de em programa de Rife, Clark, Zappicator*
- int `qualPrograma` = 0
 - identifica programa, 1-clark, 2-zappic, 3-rife, etc)*
- int `estagio` = 0
 - indica em que estagio esta - qual par de frequencias*
- int `quantosEstagios` = 0
 - indica quantos estagios-fases tem o protocolo*
- float `pwmFrequencia` = 0
 - indica a (pwm) frequencia*
- float `pwmDuracao` = 0
 - indica (pwm) a duracao do protocolo-fase-estagio*
- bool `primeiraVez` = true
 - indica se comecou(1 vez) CLARK*
- unsigned long `protocoloAntesRasc`
 - do protocolo CLARK antes rascunho*
- unsigned long `protocoloSegundosTotal` = 0
 - do protocolo CLARK contador segundos total*
- unsigned long `protocoloSegundos` = 0
 - do protocolo CLARK contador segundos*
- int `qualRife` = 1
 - indica qual protocolo Rife*
- uint32_t `frequenciaRasc` [7]
 - Operacao Manual (Rife) tabela de rascunho das frequencias.*
- uint32_t `duracaoRasc` [7]
 - Operacao Manual (Rife) tabela de duracao das frequencias.*
- const unsigned long `intervaloLeituraIna219` = 10000
 - intervalo (de 10 seg). em milissegundos de leitura do ina219*
- const int `liga` = HIGH
 - constante de acao liga (HIGH -> liga)*
- const int `desliga` = LOW
 - constante de acao desliga (LOW -> desliga)*
- const int `buzina` = 4
 - constante bit buzina = 4*
- const int `clark` = 1
 - Protocolo Clark 30KHz => 7m-20m-7m-20m-7m.*
- const int `rifePre` = 3
 - Rife prca-definidos (3 exemplos)*
- const int `zapp` = 2

- ZAPPICATOR (padrao ca 1KHZ durante 30m)*
- const int `rifeETDFL` = 4
Rife entrando Numero do Protocolo.
- const int `rifeManual` = 5
digitando frequencias e duracao (atca 6 freq.)
- const int `maximoRife` = 268
e o numero de protocolos existentes na tabela tabela_rife.h
- const uint8_t `freq_var_canal` = 0
constantes no ESP32
- const uint8_t `freq_1K_canal` = 1
uint8_t frequency 1 k canal
- const uint8_t `buz_canal` = 2
uint8_t buz canal
- const uint8_t `duty_todos` = 1
uint8_t 50% duty cycle para todos > 1000Hz
- const uint8_t `duty_baixo` = 8
uint8_t 8 bits => 256 p/freq baixas
- const int `clark_rife` = 27
int clark rife
- const int `zappicator` = 26
int zappicator
- const uint32_t `freq_inicial` = 1000
freq. Inicial => 1KHz
- uint32_t `freq_var` = 0
uint32_t frequencia variavel de rascunho
- const float `tab_clark` [11] = { 5,30000,420,0,1200,30000,420,0,1200,30000,420 }
linha da CLARK {n,x,y,x,y,x,y,x,y,x,y,x,y,.....}
- const float `tab_clarkTT` [11] = { 5,30000,7,0,20,30000,7,0,20,30000,7 }
TESTE CLARK - tempo reduzido para alguns segundos.
- const float `tab_zappic` [3] = { 1,1000,900 }
linha da zappicator {n, x,y,x,y,x,y,x,y,x,y,x,y,.....}
- const float `tab_zappicTT` [3] = { 1,1000,10 }
TESTE ZAPPICATOR.
- const float `tab_zappicTT1` [3] = { 1,1000,10 }
- const float `tab_zappicTT2` [3] = { 1,2500,10 }
- const float `tab_zappicTT3` [3] = { 1,428,10 }
- const float `tab_zappicTT4` [3] = { 1,529,10 }

8.5.1 Detailed Description

Modulo Esp32-zapper v 9.0 funcoes de tratamento do PWM para zapper, clark, rife, zappicator. funcoes da tela HMI - Nextion. funcoes de leitura do INA219 - sensor tensao e corrente.

Author

MLTF

Date

17/07/2018 - 10/12/2019

8.5.2 Variable Documentation

8.5.2.1 DEBUGANDO

```
bool DEBUGANDO = true
```

Variavel Global - boolean true indica que debug habilitado.

8.5.2.2 tab_clark

```
const float tab_clark[11] = { 5,30000,420,0,1200,30000,420,0,1200,30000,420 }
```

linha da CLARK {n,x,y,x,y,x,y,....}

8.5.2.3 tab_zappic

```
const float tab_zappic[3] = { 1,1000,900 }
```

linha da zappicator {n, x,y,x,y,x,y,....}